Chapter 18. Accessing Kubernetes from Common Programming Languages

Though most of this book is dedicated to using declarative YAML configurations, either directly via `kubectl` or through tools like Helm, there are situations when it is necessary to interact with the Kubernetes API directly from a programming language. For example, the authors of the Helm tool itself needed to write that application in a programming language. More generally, this is common if you need to write some additional tool, like a `kubectl` plug-in, or a more complex piece of code, like a Kubernetes operator.

Much of the Kubernetes ecosystem is written in the Go programming language. As a result, the Go language has the richest and most extensive client. However, there are a high-quality clients for most common programming languages (and even some uncommon ones as well). Because there is already so much documentation and so many examples of how to use the Go client, this chapter will cover the basics of interacting with the Kubernetes API server with examples in Python, Java, and .NET.

The Kubernetes API: A Client's Perspective

At the end of the day, the Kubernetes API server is just an HTTP(S) server and that is exactly how each client library perceives it, though each client has a lot of additional logic that implements the various API calls and serializes to and from JSON. Given this, you might be tempted to simply use a plain HTTP client to work with the Kubernetes APIs, but the client libraries wrap these various HTTP calls into meaningful APIs that make your code more readable (e.g., `readNamespacedPod(...)`), and meaningful typed object-models that facilitate static type checking and therefore result in fewer bugs (e.g., `Deployment`). Perhaps more importantly, the client libraries also implement Kubernetes-specific capabilities, like loading authorization information from a *kubeconfig* file or from a Pod's environment. The clients also provide implementations of the non-RESTful parts of the Kubernetes API surface area like port-forward, logs, and watches. We'll describe these advanced capabilities in later sections.

OpenAPI and Generated Client Libraries

The set of resources and functions in the Kubernetes API is huge. There are many different resources in different API groups and many different operations on each of these resources. Keeping up with all of these different resources and resource versions would be a massive (and unmistakably boring) undertaking if developers had to hand-author all of these API calls. Especially when considering that clients have to be handwritten across each of the programming languages. Instead, the clients take a different approach, and the basics of interacting with the Kubernetes API server are all generated by a computer program that is sort of like a compiler in reverse. The code generator for the API clients takes a data specification for the Kubernetes API and uses this specification to generate a client for a specific language.

The Kubernetes API is expressed in a format known as OpenAPI, which is the most common schema for representing RESTful APIs. To give you a sense of the size of the Kubernetes API, the OpenAPI specification found on GitHub is over four megabytes in size. That's a pretty big text file! The official Kubernetes client libraries are all generated using the same core code generation logic, which can be found on GitHub. It is unlikely that you will actually have to generate the client libraries yourself, but nonetheless, it is useful to understand the process by which these libraries are created. In particular, because most of the client code is generated, updates and fixes can't be made directly in the generated client code, since it would be overwritten the next time the API was generated. Instead, when an error in a client is found, fixes need to be made to either the OpenAPI specification (if the error is in the specification itself) or the code generator (if the error is in the generated code). Although this process can seem excessively complex, it is the only way that a small number of Kubernetes client authors can keep up with the breadth of the Kubernetes API.

But What About kubectl x?

When you start implementing your own logic for interacting with the Kubernetes API, it probably won't be long before you find yourself asking how to do `kubectl x`. Most people start with the `kubectl` tool when they learn Kubernetes and consequently expect that there is a 1-1 mapping between the capabilities in `kubectl` and the Kubernetes API. While some commands are directly represented in the Kubernetes API (e.g., `kubectl get pods`), most of the more sophisticated features are actually a larger number of API calls with complex logic in the `kubectl` tool.

This balance between client-side and server-side features has been a design trade-off since the beginning of Kubernetes. Many features that are now present in the API server

began as client-side implementations in `kubectl`. For example, the rollout capabilities now implemented on the server by the Deployment resource were previously implemented in the client. Likewise, until recently, `kubectl apply ...` was only available within the command-line tool, but was migrated to the server as the server-side `apply` capabilities that will be discussed later in this chapter.

Despite the general trajectory toward server-side implementations, there are still significant capabilities that remain in the client. Each of these capabilities must be reimplemented in each client library. Parity with the `kubectl` command line tool varies between languages. The Java client in particular has built a thick client that emulates much of the `kubectl` functionality.

If you can't find the functionality that you are looking for in your client library, a useful trick is to add the `--v=10` flag to your `kubectl` command. That will turn on verbose logging, including all of the HTTP requests and responses sent to the Kubernetes API server. You can use this logging to reconstruct much of what `kubectl` is doing. If you still need to dig deeper, the `kubectl` source code is also available within the Kubernetes repository.

Programming the Kubernetes API

Now you have a deeper perspective about how the Kubernetes API works and how the client and server interact. In the following sections, we'll go through how to authenticate to the Kubernetes API server and interact with resources. We'll close with advanced topics from writing operators to interacting with Pods for interactive operations.

Installing the Client Libraries

Before you can start programming with the Kubernetes API, you need to find the client libraries. We will be using the official client libraries produced by the Kubernetes project itself, though there are also a number of high-quality clients developed as independent projects. The client libraries are all hosted under the kubernetes-client repository on GitHub:

- Python
- Java
- .NET

Each of these projects features a compatibility matrix to show which versions of the client work with which versions of the Kubernetes API and also give instructions for installing the libraries using the package managers (e.g., `npm`) associated with a particular programming language.[1]

Authenticating to the Kubernetes API

The Kubernetes API server wouldn't be very safe if it allowed anyone in the world to access it and read or write the resources that it orchestrates. Consequently, the first step in programming the Kubernetes API is connecting to it and identifying yourself for authentication. Because the API server is an HTTP server at its core, these methods of authentication are core HTTP authentication methods. The very first implementations of Kubernetes used basic HTTP authentication via a user and password combination, but this approach has been deprecated in favor of more modern authentication infrastructure.

If you have been using the `kubectl` command-line tool for your interactions with Kubernetes, you may not have considered the implementation details of authentication. Fortunately, the client libraries generally make it easy to connect to the API. However, a basic understanding of how Kubernetes authentication works is still useful for debugging when things go wrong.

There are two basic ways that the `kubectl` tool and clients obtain authentication information: from a kubeconfig file and from the context of a Pod within the Kubernetes cluster.

Code that is not running inside a Kubernetes cluster requires a kubeconfig file to provide the necessary information for authentication. By default, the client searches for this file in *${HOME}/.kube/config* or the `$KUBECONFIG` environment variables. If the `KUBECONFIG` variable is present, it takes precedence over any config file located in the default home location. The kubeconfig file contains all of the information necessary to access the Kubernetes API server. The clients all have easy-to-use calls to create a client either from the default locations or from a kubeconfig file supplied in the code itself:

Python

```
config.load_kube_config()
```

Java

```
ApiClient client = Config.defaultClient();
Configuration.setDefaultApiClient(client);
```

.NET

```
var config = KubernetesClientConfiguration.BuildDefaultConfig();
var client = new Kubernetes(config);
```

NOTE

Authentication for many cloud providers occurs via an external executable that knows how to generate a token for the Kubernetes cluster. This executable is often installed as part of the cloud provider's command-line tooling. When you write code to interact with the Kubernetes API, you need to make sure that this executable is also available in the context where the code is running so that it can be executed to obtain the token.

Within the context of a Pod in a Kubernetes cluster, the code running in the Pod has access to a Kubernetes service account that is associated with that Pod. The files containing the relevant token and certificate authority are placed into the Pod by Kubernetes as a volume when the Pod is created. Within a Kubernetes cluster, the API server is always available at a fixed DNS name, generally `kubernetes`. Because all of the necessary data is present in the Pod, a kubeconfig file is unnecessary and the client can synthesize its configuration from its context. The clients all have easy-to-use calls to create such an "in cluster" client:

Python

```
config.load_incluster_config()
```

Java

```
ApiClient client = ClientBuilder.cluster().build();
Configuration.setDefaultApiClient(client);
```

.NET

```
var config = KubernetesClientConfiguration.InClusterConfig()
var client = new Kubernetes(config);
```

NOTE

The default service account associated with Pods has minimal roles (RBAC) granted to it. This means that by default, the code running in a Pod can't do much with the Kubernetes API. If you are getting authorization errors, you may need to adjust the service account to one that is specific to your code and has access to the necessary roles in the cluster.

Accessing the Kubernetes API

The most common ways that people interact with the Kubernetes API is via basic operations like creating, listing, and deleting resources. Because all of the clients are generated from the same OpenAPI specification, they all follow the same rough pattern. Before diving into the code, there are a couple more details of the Kubernetes API that are necessary to understand.

In Kubernetes, there is a distinction between namespaced and cluster-level resources. *Namespaced* resources exist within a Kubernetes namespace; for example, a Pod or Deployment may exist in the `kube-system` namespace. *Cluster-level* resources exist only once throughout the entire cluster. The most obvious example of such a resource is a Namespace, but other cluster-level resources include CustomResourceDefinitions and ClusterRoleBindings. This distinction is important because it is preserved in the function calls that you use to access the resources. For example, to list Pods in the `default` namespace in Python, you would write `api.list_namespaced_pods('default')`. To list Namespaces, you would write `api.list_namespaces()`.

The second concept you need to understand is an *API group*. In Kubernetes, all of the resources are grouped into different sets of APIs. This is largely hidden from users of the `kubectl` tool, though you may have seen it within the `apiVersion` field in a YAML specification of a Kubernetes object. When programming against the Kubernetes API, this grouping becomes important, because often each API group has its own client for interacting with that set of resources. For example, to create a client to interact with a Deployment resource (which exists in the `apps/v1` API group and version) you create a `new AppsV1Api()` object that knows how to interact with all resources in the `apps/v1` API group and version. An example of how to create a client for an API group is shown in the following section.

Putting It All Together: Listing and Creating Pods in Python, Java, and .NET

We're now ready to actually write some code. Begin by creating a client object, then use that to list the Pods in the "default" namespace; here is code to do that in Python, Java, and .NET:

Python

```
config.load_kube_config()
api = client.CoreV1Api()
pod_list = api.list_namespaced_pod('default')
```

Java

```
ApiClient client = Config.defaultClient();
Configuration.setDefaultApiClient(client);
CoreV1Api api = new CoreV1Api();
V1PodList list = api.listNamespacedPod("default");
```

.NET

```
var config = KubernetesClientConfiguration.BuildDefaultConfig();
var client = new Kubernetes(config);
var list = client.ListNamespacedPod("default");
```

Once you have figured out how to list, read, and delete objects, the next common task is creating new objects. The API call to create the object is easy enough to figure out (e.g., `create_namespaced_pod` in Python), but actually defining the new Pod resources can be more complicated.

Here's how you create a Pod in Python, Java, and .NET:

Python

```
container = client.V1Container(
    name="myapp",
    image="my_cool_image:v1",
)
```

```
pod = client.V1Pod(
    metadata = client.V1ObjectMeta(
      name="myapp",
    ),
    spec=client.V1PodSpec(containers=[container]),
)
```

Java

```
V1Pod pod =
    new V1PodBuilder()
        .withNewMetadata().withName("myapp").endMetadata()
        .withNewSpec()
          .addNewContainer()
            .withName("myapp")
            .withImage("my_cool_image:v1")
          .endContainer()
        .endSpec()
        .build();
```

.NET

```
var pod = new V1Pod()
{
    Metadata = new V1ObjectMeta{ Name = "myapp", },
    Spec = new V1PodSpec
    {
        Containers = new[] {
          new V1Container() {
            Name = "myapp", Image = "my_cool_image:v1",
          },
        },
    }
 };
```

Creating and Patching Objects

When you explore the client API for Kubernetes, you will notice that there are seemingly three different ways to manipulate resources, namely `create`, `replace`, and `patch`. These three verbs represent slightly different semantics for interacting with resources:

Create

As you can tell from the name, this creates a new resource. However, it will fail if the resource already exists.

Replace

This replaces an existing resource completely, without looking at the existing resource. When you use `replace`, you have to specify a complete resource.

Patch

This modifies an existing resource, leaving untouched the parts of the resource that did not change. When using `patch`, you use a special Patch resource rather than sending the resource (e.g., the Pod) that you are modifying.

NOTE

Patching a resource can be complicated. In many cases, it is easier to just replace it. However, in some cases, especially with large resources, patching the resource can be much more efficient in terms of network bandwidth and API server processing. Additionally, multiple actors can patch different parts of the resource simultaneously without worrying about write conflicts, which reduces overhead.

To patch a Kubernetes resource, you have to create a Patch object representing the change that you want to make to the resource. There are three formats for this patch supported by Kubernetes: JSON Patch, JSON Merge Patch, and strategic merge patch. The first two patch formats are RFC standards used in other places, and the third is a Kubernetes-developed patch format. Each of the patch formats has advantages and disadvantages. In these examples, we will use JSON Patch because it is the simplest to understand.

Here's how you patch a Deployment to increase the replicas to three:

Python

```
deployment.spec.replicas = 3

api_response = api_instance.patch_namespaced_deployment(
  name="my-deployment",
  namespace="some-namespace",
  body=deployment)
```

Java

```
// JSON-patch format
static String jsonPatch =
  "[{\"op\":\"replace\",\"path\":\"/spec/replicas\",\"value\":3}]";

V1Deployment patched =
        PatchUtils.patch(
           V1Deployment.class,
           () ->
              api.patchNamespacedDeploymentCall(
                 "my-deployment",
                 "some-namespace",
                 new V1Patch(jsonPatchStr),
                 null,
                 null,
                 null,
                 null,
                 null),
           V1Patch.PATCH_FORMAT_JSON_PATCH,
           api.getApiClient());
```

.NET

```
var jsonPatch = @"
[{
   ""op"": ""replace"",
   ""path"": ""/spec/replicas"",
   ""value"": 3
}]";

client.PatchNamespacedPod(
  new V1Patch(patchStr, V1Patch.PatchType.JsonPatch),
  "my-deployment",
  "some-namespace");
```

In each of these code samples, the Deployment resource has been patched to set the number of replicas in the deployment to three.

Watching Kubernetes APIs for Changes

Resources in Kubernetes are declarative. They represent the desired state of the system. To make that desired state a reality, a program must watch the desired state for changes and take action to make the current state of the world match the desired state.

Because of this pattern, one of the most common tasks when programming against the Kubernetes API is to watch for changes to a resource and then take some action based on those changes. The easiest way to do this is through polling. *Polling* simply calls the list function described above at a constant interval (such as every 60 seconds) and enumerates all of the resources that the code is interested in. While this code is easy to write, it has numerous drawbacks for both the client code and the API server. Polling introduces unnecessary latency, since waiting for the polling cycle to come around introduces delays for changes that occur just after the previous poll completed. Additionally, polling causes heavier load on the API server because it repeatedly returns resources that haven't changed. While many simple clients begin by using polling, too many clients polling the API server can overload it and add latency.

To solve this problem, the Kuberentes API also provides *watch*, or event-based, semantics. Using a `watch` call, you can register interest in specific changes with the API server and, instead of repeatedly polling, the API server will send notifications whenever a change occurs. In practical terms, the client performs a hanging GET to the HTTP API server. The TCP connection that underlies this HTTP request stays open for the duration of the watch, and the server writes a response to that stream (but does not close the stream) whenever a change occurs.

From a programmatic perspective, `watch` semantics enable event-based programming, changing a `while` loop that repeatedly polls into a collection of callbacks. Here are examples of watching Pods for changes:

Python

```
config.load_kube_config()
api = client.CoreV1Api()
w = watch.Watch()

for event in w.stream(v1.list_namespaced_pods, "some-namespace"):
  print(event)
```

Java

```
    ApiClient client = Config.defaultClient();
    CoreV1Api api = new CoreV1Api();

    Watch<V1Namespace> watch =
      Watch.createWatch(
        client,
        api.listNamespacedPodCall(
          "some-namespace",
          null,
          null,
          null,
          null,
          null,
          Integer.MAX_VALUE,
          null,
          null,
          60,
          Boolean.TRUE);
        new TypeToken<Watch.Response<V1Pod>>() {}.getType());

    try {
     for (Watch.Response<V1Pod> item : watch) {
      System.out.printf(
        "%s : %s%n", item.type, item.object.getMetadata().getName());
     }
    } finally {
     watch.close();
    }
```

.NET

```
var config = KubernetesClientConfiguration.BuildConfigFromConfigFile();
var client = new Kubernetes(config);

var watch =
 client.ListNamespacedPodWithHttpMessagesAsync("default", watch: true);
using (watch.Watch<V1Pod, V1PodList>((type, item) =>
{
 Console.WriteLine(item);
}
```

In each of these examples, rather than a repetitive polling loop, the watch API call delivers each change to a resource to a callback provided by the user. This both reduces la-

tency and load on the Kubernetes API server.

Interacting with Pods

The Kubernetes API also provides functions for directly interacting with the applications running in a Kubernetes Pod. The `kubectl` tool provides a number of commands for interacting with Pods, namely `logs`, `exec`, and `port-forward`, and it is possible to use each of these from within custom code as well.

NOTE

Because the `logs`, `exec`, and `port-forward` APIs are nonstandard in a RESTful sense, they require custom logic in the client libraries and are thus somewhat less consistent between the different clients. Unfortunately, there is no option other than learning the implementation for each language.

When getting the logs for a Pod, you have to decide if you are going to read the Pod logs to get a snapshot of their current state or if you are going to stream them to receive new logs as they happen. If you stream the logs (the equivalent of `kubectl logs -f ...`), you create an open connection to the API server, and new log lines are written to this stream as they are written to the Pod. If not, you simply receive the current contents of the logs.

Here's how you both read and stream the logs:

Python

```
config.load_kube_config()
api = client.CoreV1Api()
log = api_instance.read_namespaced_pod_log(
  name="my-pod", namespace="some-namespace")
```

Java

```
V1Pod pod = ...; // some code to define or get a Pod here
PodLogs logs = new PodLogs();
InputStream is = logs.streamNamespacedPodLog(pod);
```

.NET

```
IKubernetes client = new Kubernetes(config);
var response = await client.ReadNamespacedPodLogWithHttpMessagesAsync(
   "my-pod", "my-namespace", follow: true);
var stream = response.Body;
```

Another common task is to execute some command within a Pod and get the output of running that task. You can use the `kubectl exec ...` command on the command line. Under the hood, the API that implements this is creating a WebSocket connection to the API server. WebSockets enable multiple streams of data (in this case, `stdin`, `stdout`, and `stderr`) to coexist on the same HTTP connection. If you've never had experience with WebSockets, don't worry; the details of interacting with WebSockets are handled by the client libraries.

Here's how you execute the `ls /foo` command in a Pod:

Python

```
cmd = [ 'ls', '/foo' ]
response = stream(
   api_instance.connect_get_namespaced_pod_exec,
   "my-pod",
   "some-namespace",
   command=cmd,
   stderr=True,
   stdin=False,
   stdout=True,
   tty=False)
```

Java

```
ApiClient client = Config.defaultClient();
Configuration.setDefaultApiClient(client);
Exec exec = new Exec();
final Process proc =
 exec.exec("some-namespace",
        "my-pod",
        new String[] {"ls", "/foo"},
        true,
        true /*tty*/);
```

.NET

```
var config = KubernetesClientConfiguration.BuildConfigFromConfigFile();
IKubernetes client = new Kubernetes(config);
var webSocket =
    await client.WebSocketNamespacedPodExecAsync(
     "my-pod", "some-namespace", "ls /foo", "my-container-name");
var demux = new StreamDemuxer(webSocket);
demux.Start();
var stream = demux.GetStream(1, 1);
```

In addition to running commands in a Pod, you can also port-forward network connections from a Pod to code running on the local machine. Like `exec`, the port-forwarded traffic goes over a WebSocket. It is up to your code what it does with this port-forwarded socket. You could simply send a single request and receive a response as a string of bytes, or you could build a complete proxy server (like what `kubectl port-forward` does) to serve arbitrary requests through this proxy.

Regardless of what you intend to do with the connection, here's how you set up port-forwarding:

Python

```
pf = portforward(
    api_instance.connect_get_namespaced_pod_portforward,
    'my-pod', 'some-namespace',
    ports='8080',
)
```

Java

```
PortForward fwd = new PortForward();

List<Integer> ports = new ArrayList<>();
int localPort = 8080;
int targetPort = 8080;
ports.add(targetPort);
final PortForward.PortForwardResult result =
    fwd.forward("some-namespace", "my-pod", ports);
```

.NET

```
var config = KubernetesClientConfiguration.BuildConfigFromConfigFile();
IKubernetes client = new Kubernetes(config);
var webSocket = await client.WebSocketNamespacedPodPortForwardAsync(
  "some-namespace", "my-pod", new int[] {8080}, "v4.channel.k8s.io");
var demux = new StreamDemuxer(webSocket, StreamType.PortForward);
demux.Start();
var stream = demux.GetStream((byte?)0, (byte?)0);
```

Each of these examples creates a connection from port 8080 in a Pod to port 8080 in your program. The code returns the byte streams necessary, communicating across this port-forwarding channel. You can use these streams for sending and receiving messages.

Summary

The Kubernetes API provides rich and powerful functionality for you to write custom code. Writing your applications in the language that best suits a task or a persona shares the power of the orchestration API with as many Kubernetes users as possible. When you're ready to move beyond scripting calls to the `kubectl` executable, the Kubernetes client libraries provide a way to dive deep into the API to build an operator, a monitoring agent, a new user interface, or whatever your imagination can dream up.

1 We did not include JavaScript examples for brevity, but it is also actively developed.

Previous chapter
17. Extending Kubernetes

Next chapter
19. Securing Applications in Kubernetes